

Practical No. 06

ON

SSCA4030 - Image Processing

TITLE: Fundamentals of Image Preprocessing

**BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)**

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

1. OBJECTIVE

The objective of this practical is to understand and implement fundamental image preprocessing techniques used in image processing and computer vision. The practical focuses on improving image quality, modifying intensity distribution, reducing noise, and preparing images for further image analysis, machine learning, and deep learning applications.

2. TOOLS AND TECHNOLOGIES USED

- **Python 3.x** – Used for implementing the image preprocessing operations.
- **Google Colab** – Used as the development and execution environment.
- **OpenCV (cv2)** – Used for image loading, color conversion, resizing, filtering, thresholding, and morphological operations.
- **NumPy** – Used for numerical and pixel-level image processing.
- **Matplotlib** – Used for displaying images and generating histograms.

3. IMPLEMENTATION AND RESULTS

```
[1]
✓ 0s
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")

Libraries imported successfully!
```

```
[2]
✓ 15s
from google.colab import files

uploaded = files.upload()

Choose Files my_image.jpg
my_image.jpg(image/jpeg) - 118826 bytes, last modified: 7/7/2026 - 100% done
Saving my_image.jpg to my_image.jpg
```

```
[3]
✓ 0s
# Get the uploaded filename automatically
filename = next(iter(uploaded))

# Read the image using OpenCV
image = cv2.imread(filename)

if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image uploaded successfully!")
    print("Filename:", filename)

Image uploaded successfully!
Filename: my_image.jpg
```

```
[4]
✓ 1s # Convert BGR to RGB for displaying with Matplotlib
      rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

      plt.figure(figsize=(8, 5))
      plt.imshow(rgb_image)
      plt.title("Original Uploaded Image")
      plt.axis("off")
      plt.show()
```



Original Uploaded Image

Task 1 – Image Loading and Inspection

```
[5]
✓ 0s # Get image dimensions
      height, width = image.shape[:2]

      # Get number of channels
      if len(image.shape) == 2:
          channels = 1
      else:
          channels = image.shape[2]

      print("IMAGE INFORMATION")
      print("-----")
      print("Filename:", filename)
      print("Width:", width)
      print("Height:", height)
      print("Channels:", channels)
      print("Data Type:", image.dtype)
      print("Minimum Pixel Value:", image.min())
      print("Maximum Pixel Value:", image.max())
```

```
IMAGE INFORMATION
-----
Filename: my_image.jpg
Width: 736
Height: 552
Channels: 3
Data Type: uint8
Minimum Pixel Value: 0
Maximum Pixel Value: 255
```

Task 2 – Color Space Conversion

```
[6]
✓ 3s # Convert BGR to RGB
      rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

      # Convert BGR to grayscale
      gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

      # Display all versions
      plt.figure(figsize=(15, 5))

      plt.subplot(1, 3, 1)
      plt.imshow(image)
      plt.title("BGR Image")
      plt.axis("off")

      plt.subplot(1, 3, 2)
      plt.imshow(rgb_image)
      plt.title("RGB Image")
      plt.axis("off")

      plt.subplot(1, 3, 3)
      plt.imshow(gray_image, cmap="gray")
      plt.title("Grayscale Image")
      plt.axis("off")

      plt.tight_layout()
      plt.show()
```



Task 3 – Image Resizing

```
[7]
✓ 3s
# Resize image to 224 x 224
resize_224 = cv2.resize(
    image,
    (224, 224),
    interpolation=cv2.INTER_LINEAR
)

# Resize image to 128 x 128
resize_128 = cv2.resize(
    image,
    (128, 128),
    interpolation=cv2.INTER_LINEAR
)

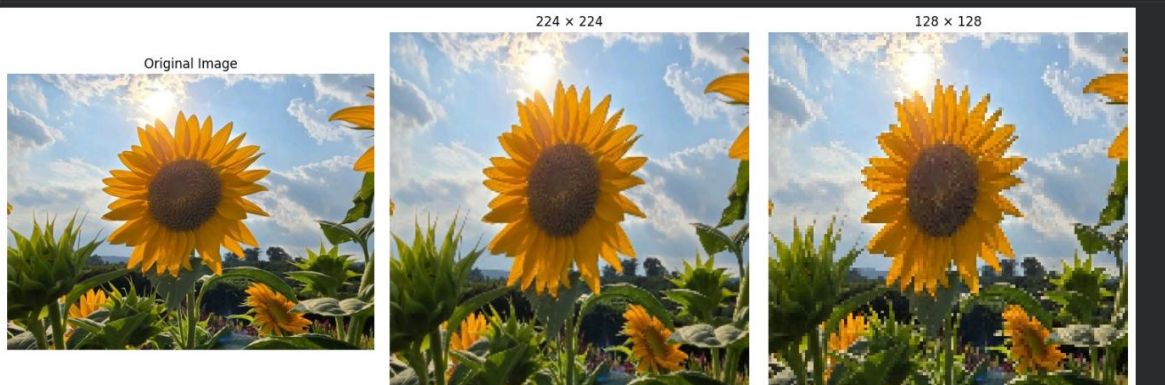
# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(rgb_image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(resize_224, cv2.COLOR_BGR2RGB))
plt.title("224 x 224")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(resize_128, cv2.COLOR_BGR2RGB))
plt.title("128 x 128")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 4 – Cropping / Region of Interest (ROI)

```
[8]
✓ 4s
height, width = image.shape[:2]

# Define center region
x1 = width // 4
y1 = height // 4
x2 = 3 * width // 4
y2 = 3 * height // 4

# Crop the image
roi = image[y1:y2, x1:x2]

# Display original and cropped image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(rgb_image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Cropped ROI")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 5 – Image Normalization

```
[9]
✓ 0s
# Convert image to floating point
normalized_image = image.astype(np.float32) / 255.0

print("Original Image Range:")
print("Minimum:", image.min())
print("Maximum:", image.max())

print("\nNormalized Image Range:")
print("Minimum:", normalized_image.min())
print("Maximum:", normalized_image.max())
```

```
Original Image Range:
Minimum: 0
Maximum: 255

Normalized Image Range:
Minimum: 0.0
Maximum: 1.0
```

```
[10]
✓ 4s
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(rgb_image)
plt.title("Original [0,255]")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(normalized_image)
plt.title("Normalized [0,1]")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 6 – Histogram Equalization

```
[11] ✓ 2s
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

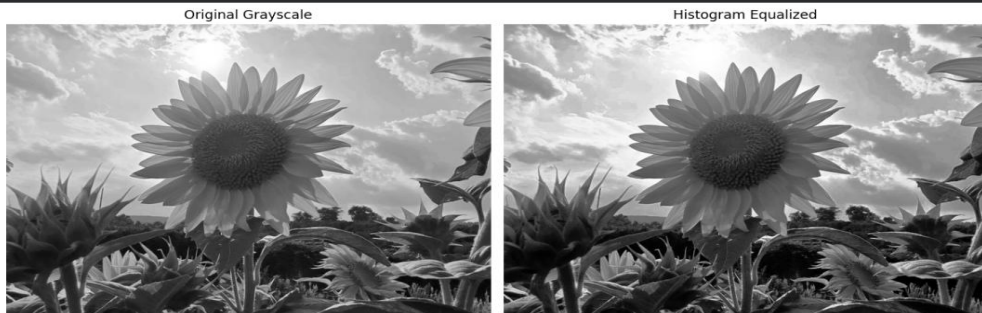
# Apply histogram equalization
equalized = cv2.equalizeHist(gray)

# Display images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized")
plt.axis("off")

plt.tight_layout()
plt.show()
```

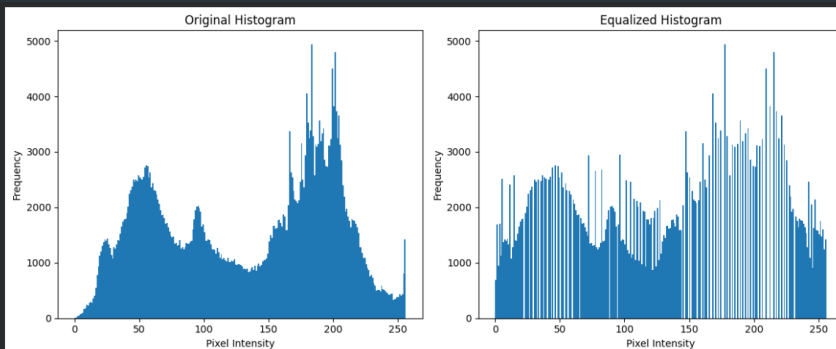


```
[12] ✓ 1s
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(gray.ravel(), bins=256, range=[0, 256])
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.subplot(1, 2, 2)
plt.hist(equalized.ravel(), bins=256, range=[0, 256])
plt.title("Equalized Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



Task 7 – Noise Addition

```
[13] ✓
# Convert image to float
image_float = image.astype(np.float32)

# Gaussian noise parameters
mean = 0
sigma = 25

# Generate Gaussian noise
gaussian_noise = np.random.normal(
    mean,
    sigma,
    image.shape
)

# Add noise
gaussian_noisy = image_float + gaussian_noise

# Keep values between 0 and 255
gaussian_noisy = np.clip(
    gaussian_noisy,
    0,
    255
).astype(np.uint8)
```

```
[14] ✓
salt_pepper = image.copy()

amount = 0.03

num_pixels = int(
    amount * image.shape[0] * image.shape[1]
)

# Add salt noise
for _ in range(num_pixels // 2):
    y = np.random.randint(0, image.shape[0])
    x = np.random.randint(0, image.shape[1])
    salt_pepper[y, x] = [255, 255, 255]

# Add pepper noise
for _ in range(num_pixels // 2):
    y = np.random.randint(0, image.shape[0])
    x = np.random.randint(0, image.shape[1])
    salt_pepper[y, x] = [0, 0, 0]
```

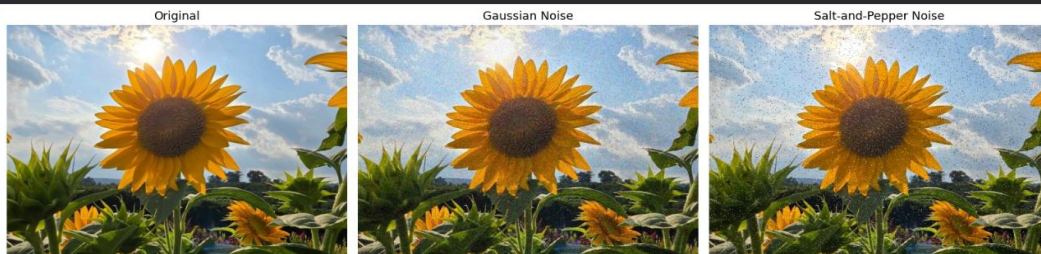
```
[15] ✓ 3s
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(rgb_image)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(gaussian_noisy, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Noise")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(salt_pepper, cv2.COLOR_BGR2RGB))
plt.title("Salt-and-Pepper Noise")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 8 – Image Filtering

```
[16] ✓
# Mean / Average filter
mean_filtered = cv2.blur(
    salt_pepper,
    (5, 5)
)

# Gaussian filter
gaussian_filtered = cv2.GaussianBlur(
    salt_pepper,
    (5, 5),
    0
)

# Median filter
median_filtered = cv2.medianBlur(
    salt_pepper,
    5
)
```

```
[17] ✓ 1s
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(mean_filtered, cv2.COLOR_BGR2RGB))
plt.title("Mean Filter")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(gaussian_filtered, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(median_filtered, cv2.COLOR_BGR2RGB))
plt.title("Median Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 9 – Thresholding

```
[18] ✓ Os
# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Fixed binary threshold
threshold_value = 127

_, binary_threshold = cv2.threshold(
    gray,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

# Otsu thresholding
otsu_value, otsu_threshold = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Fixed Threshold:", threshold_value)
print("Otsu Threshold:", otsu_value)
```

Fixed Threshold: 127
Otsu Threshold: 126.0

```
[19] ✓ h
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(binary_threshold, cmap="gray")
plt.title("Binary Threshold")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Threshold")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Task 10 – Morphological Processing

```
[20] ✓
kernel = np.ones((5, 5), np.uint8)
```

```
[21] ✓
erosion = cv2.erode(
    binary_threshold,
    kernel,
    iterations=1
)
```

```
[22] ✓
dilation = cv2.dilate(
    binary_threshold,
    kernel,
    iterations=1
)
```

```
[23] ✓
opening = cv2.morphologyEx(
    binary_threshold,
    cv2.MORPH_OPEN,
    kernel
)
```

```
[24] ✓
closing = cv2.morphologyEx(
    binary_threshold,
    cv2.MORPH_CLOSE,
    kernel
)
```




4. COMPARATIVE ANALYSIS

4.1 Mean Filter vs Gaussian Filter vs Median Filter

The mean filter smooths the image by averaging neighboring pixels, while the Gaussian filter applies weighted smoothing. The median filter is more effective for removing salt-and-pepper noise while preserving important edges.

4.2 Original Image vs Histogram Equalization

Histogram equalization improves image contrast by redistributing pixel intensity values. It can make details more visible, especially in low-contrast images.

4.3 Fixed Threshold vs Otsu Threshold

Fixed thresholding uses a manually selected threshold value, whereas Otsu thresholding automatically determines a suitable threshold from the image intensity distribution.

4.4 Erosion vs Dilation / Opening vs Closing

Erosion reduces foreground regions, while dilation expands them. Opening helps remove small foreground noise, whereas closing helps fill small holes and gaps.

4.5 Original Resolution vs 224×224 Image

The original image preserves more spatial details, while resizing to 224×224 provides a fixed image size suitable for many computer vision and deep learning applications. Resizing can reduce fine details and may cause distortion if the aspect ratio is not maintained.

5. REAL-LIFE APPLICATION QUESTIONS

Q1. A CCTV camera produces images with significant salt-and-pepper noise. Which filter would you select and why?

I would select a **median filter** because it is effective for removing salt-and-pepper noise while preserving important edges.

Q2. A medical image has poor contrast. Which preprocessing methods could improve visibility without changing the underlying structure?

Histogram equalization can improve contrast by redistributing pixel intensity values and making important details more visible.

Q3. Why is image normalization commonly performed before training a neural network?

Image normalization converts pixel values into a consistent numerical range, which helps provide suitable input for neural network training.

Q4. Why can excessive smoothing reduce the accuracy of an edge-detection or object-detection system?

Excessive smoothing can remove important **edges, textures, and fine details** from an image. This may reduce the accuracy of systems that depend on these features.

Q5. A document-scanning system must separate text from a white background. Which thresholding and morphological operations could be useful?

Binary thresholding or Otsu thresholding can separate text from the background. **Opening** can remove small noise, while **closing** can fill small gaps in text regions.

Q6. Why should the aspect ratio be considered when resizing an image for a CNN?

The aspect ratio should be considered because ignoring it can **distort the shape and proportions of objects**, which may negatively affect CNN performance.

Q7. Explain one situation where preprocessing can improve model performance and one situation where inappropriate preprocessing can reduce it.

Preprocessing can improve model performance when noise is removed from an image before object detection, making important features clearer. However, inappropriate preprocessing, such as excessive smoothing, can remove important edges and details and reduce model accuracy.

6. OVERALL OBSERVATIONS

The practical demonstrated that different image preprocessing techniques are useful for different image-processing requirements.

- **Color space conversion** simplifies image representation for specific processing tasks.
- **Resizing** provides a consistent image size for further processing.
- **Cropping** focuses processing on a specific Region of Interest (ROI).
- **Normalization** converts pixel values into a consistent range.
- **Histogram equalization** improves contrast in suitable images.
- **Noise addition** demonstrates the effect of unwanted variations in images.
- **Filtering** helps reduce noise while maintaining useful image information.
- **Thresholding** separates foreground and background regions.

- **Morphological operations** help modify and clean binary image structures.

7. CONCLUSION

The practical successfully demonstrated the implementation of fundamental image preprocessing techniques using Python, OpenCV, NumPy, and Matplotlib.

The experiment included image loading, color space conversion, resizing, cropping, normalization, histogram equalization, noise addition, filtering, thresholding, and morphological processing.

The results showed that each preprocessing technique has a specific purpose. Proper preprocessing can improve image quality, reduce noise, enhance contrast, and make images more suitable for further computer vision, machine learning, and deep learning tasks.

Overall, the practical provided a clear understanding of how image preprocessing affects image quality and prepares images for subsequent analysis.

8. REFERENCES

1. **OpenCV Documentation** – Image Processing and Computer Vision.
2. **NumPy Documentation** – Numerical Computing in Python.
3. **Matplotlib Documentation** – Data Visualization and Plotting.
4. **Google Colab** – Python Notebook Environment.
5. **Image Processing & Computer Vision – Image Preprocessing Assignment** – Course-provided assignment material